```
# ==========================================
# CORNER DETECTION IN TEXTURED REGIONS
# Feature Selection using Shi-Tomasi
# ==========================================

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# ------------------------------------------
# 1. Upload Image
# ------------------------------------------
uploaded = files.upload()
filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

if img is None:
    print("Error: Image could not be loaded.")
else:

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # ---------------------------------------
    # 2. Detect MANY Initial Corners
    # ---------------------------------------
    corners_many = cv2.goodFeaturesToTrack(
        gray,
        maxCorners=1000,
        qualityLevel=0.01,
        minDistance=2,
        blockSize=3
    )

    result_many = img.copy()

    if corners_many is not None:
        corners_many = np.int32(corners_many)

        for corner in corners_many:
            x, y = corner.ravel()

            cv2.circle(
                result_many,
                (x, y),
                3,
                (0, 0, 255),
                -1
            )

    # ---------------------------------------
    # 3. Improved Feature Selection
    # ---------------------------------------
    # Higher qualityLevel removes weak corners
    # Larger minDistance removes clustered points

    corners_selected = cv2.goodFeaturesToTrack(
        gray,
        maxCorners=150,
        qualityLevel=0.08,
        minDistance=15,
        blockSize=7
    )
```

```
result_selected = img.copy()

if corners_selected is not None:
    corners_selected = np.int32(
        corners_selected
    )

    for corner in corners_selected:
        x, y = corner.ravel()

        cv2.circle(
            result_selected,
            (x, y),
            5,
            (0, 255, 0),
            -1
        )

# ----------------------------------------
# 4. Spatial Grid Filtering
# ----------------------------------------
# Keeps only a limited number of points
# in each image region

final_points = []

if corners_selected is not None:

    h, w = gray.shape

    grid_rows = 6
    grid_cols = 6

    cell_h = h // grid_rows
    cell_w = w // grid_cols

    max_points_per_cell = 3

    for r in range(grid_rows):

        for c in range(grid_cols):

            cell_points = []

            for point in corners_selected:

                x, y = point.ravel()

                if (
                    c * cell_w <= x < (c + 1) * cell_w
                    and
                    r * cell_h <= y < (r + 1) * cell_h
                ):
                    cell_points.append(
                        (x, y)
                    )

            # Keep only a few points per cell
            cell_points = cell_points[
                :max_points_per_cell
            ]

            final_points.extend(
                cell_points
            )

# ----------------------------------------
# 5. Draw Final Selected Keypoints
```

```
# ----------------------------------------
result_final = img.copy()

for x, y in final_points:

    cv2.circle(
        result_final,
        (x, y),
        5,
        (0, 255, 0),
        -1
    )

# ----------------------------------------
# 6. Display Results
# ----------------------------------------

plt.figure(figsize=(16, 10))

plt.subplot(2, 2, 1)
plt.imshow(
    cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
)
plt.title("1. Textured Input Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(
    cv2.cvtColor(
        result_many,
        cv2.COLOR_BGR2RGB
    )
)
plt.title(
    "2. Initial Detection - Too Many Keypoints"
)
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(
    cv2.cvtColor(
        result_selected,
        cv2.COLOR_BGR2RGB
    )
)
plt.title(
    "3. After Quality + Distance Filtering"
)
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(
    cv2.cvtColor(
        result_final,
        cv2.COLOR_BGR2RGB
    )
)
plt.title(
    "4. Final Spatially Distributed Keypoints"
)
plt.axis("off")

plt.tight_layout()
plt.show()

# ----------------------------------------
# 7. Number of Keypoints
```

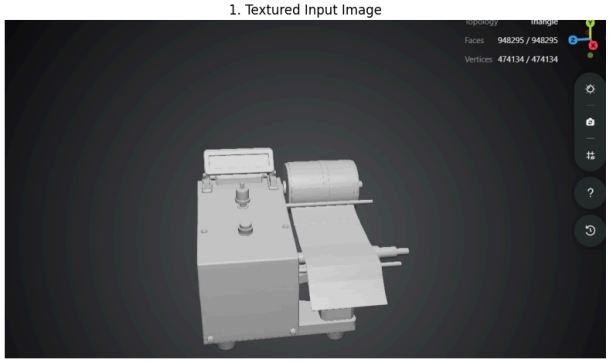
```
# ----------------------------------------

initial_count = (
    0 if corners_many is None
    else len(corners_many)
)

selected_count = (
    0 if corners_selected is None
    else len(corners_selected)
)

final_count = len(final_points)

print(
    "Initial keypoints detected :",
    initial_count
)

print(
    "After feature filtering    :",
    selected_count
)

print(
    "Final selected keypoints   :",
    final_count
)
```

Choose Files Screenshot… - Copy.png

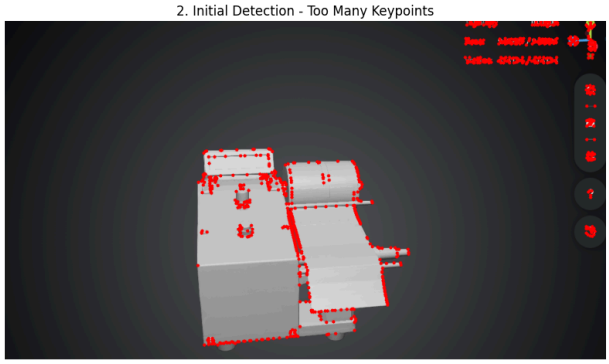
Screenshot 2026-05-12 130320 - Copy.png(image/png) - 346705 bytes, last modified: 5/12/2026 - 100% done

Saving Screenshot 2026-05-12 130320 - Copy.png to Screenshot 2026-05-12 130320 - Copy.png

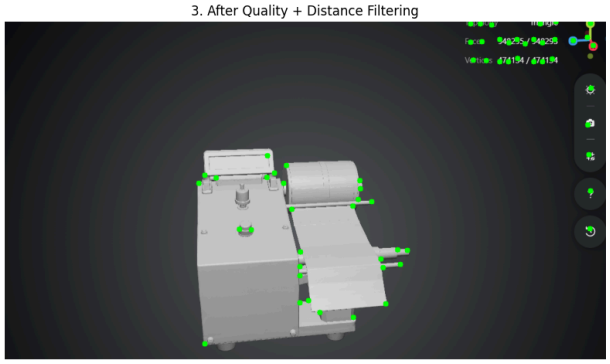
1. Textured Input Image



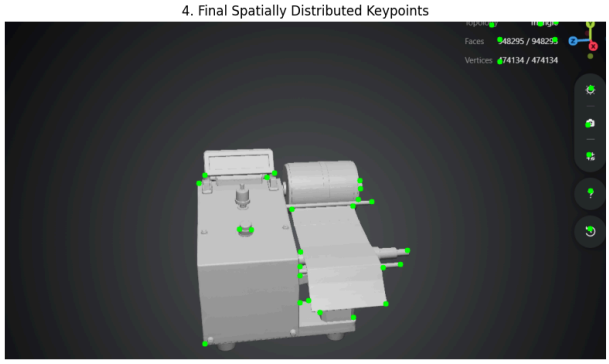
2. Initial Detection - Too Many Keypoints



3. After Quality + Distance Filtering



4. Final Spatially Distributed Keypoints



```
Initial keypoints detected : 814
After feature filtering    : 61
Final selected keypoints   : 35
```